Performance Engineering

Analysis and Optimisation of the CloverLeaf Mini-App on the HAICGU Kunpeng 920 Cluster

Jort Oosterveld S5545781 & Sergiu-Adrian Preda S6778542
Group 11

Draft, October 14, 2026

1 Introduction

CloverLeaf [2] solves the compressible Euler equations on a 2D structured grid with an explicit Lagrangian–Eulerian scheme; it is a stencil code and a standard case for memory-bound performance engineering [1]. We work on the reference implementation (`CloverLeaf_ref`, commit `b5e598d`, MPI+OpenMP) and optimise its *C kernels* (`use_c_kernels`) on one node of the HAICGU cluster. Section 2 establishes where the baseline stands against the hardware, Section 3 derives the kernel changes that can still pay off and shows them in code, and Section 4 compares the optimised kernels with the baseline at every layout.

2 Baseline Analysis

2.1 Application and system

One time step runs, in order, the equation of state (`ideal_gas`), artificial viscosity, the stable time step (`calc_dt`), a predictor and a corrector PdV step, the node acceleration, the volume fluxes, the advection of cell quantities (`advec_cell`) and of momentum (`advec_mom`) in two directional sweeps, and the reset of the time levels. Every kernel is a loop nest over the grid with a ± 1 stencil; the two advection kernels contain an upwind/limiter loop with data-dependent indices. All fields are double-precision arrays of $(n_x + 5) \times (n_y + 5)$ elements. The benchmark problem is **bm3840**: 3840^2 cells, 87 time steps, about 25 arrays of 118 MB ($\approx$3 GB working set, far beyond the 2×64 MB L3), with CloverLeaf's built-in kinetic-energy check (`PASSED` in every run reported here).

The node has 2× Kunpeng 920 (ARMv8, 2.6 GHz, 64 cores each, 4 NUMA domains of 32 cores) and 16 channels of DDR4-2666 (128 GB, 341 GB/s theoretical), under a RHEL-8-compatible Linux with 64 KB pages. Software: GCC/GFortran 14.1.0, OpenMPI 5.0.3, PAPI 7.1.0 and Linux `perf`; the kernels are compiled with the project's `GNU` settings, `-O3 -march=native -funroll-loops -fopenmp`. Nodes are used exclusively. A configuration "$r \times t$" is r MPI ranks with t OpenMP threads each. Every rank is bound to t consecutive cores (`-map-by slot:PE=`t) and OpenMP pins one thread per core (`OMP_PLACES=cores`, `OMP_PROC_BIND=close`). The Fortran kernels are run at the same configurations for reference only.

kernels	runs	deviation (%)	layouts (all identical)
c	16	0.5726209E-02	128x1, 4x32, 64x2, 1x128, 32x4
fortran	11	0.1787088E-01	128x1, 4x32, 64x2

Table 1: Baseline kinetic-energy deviation of `clover_bm.in` with the unmodified kernels, the value an optimised build has to reproduce (jobs 34729 and 34796).

2.2 Methodology

Tools. Three independent sources: (1) CloverLeaf's internal profiler (wall clock per algorithmic phase); (2) PAPI per kernel, through a small instrumentation layer (`papi_prof`) that wraps the `omp parallel` region of each C kernel, keeps one event set per thread and writes the counts per thread at exit (compiles to nothing without `-DUSE_PAPI`); cycles/instructions, instruction mix and stall sets are measured in separate runs; (3) Linux `perf`: sampling profiles, whole-run counters, and the HiSilicon DDR-controller PMU (`hisi_sccl*_ddrc*/flux_rd,wr`), read system-wide around every run, which counts the bytes that cross the memory interface per controller.

Protocol. Five repetitions per configuration, median and min–max reported; the timed region is the complete run of 87 steps including start-up; every repetition runs under the DDR-controller counters, so wall clock and hardware bytes belong to the same run. All bandwidths are *hardware bytes* (DDR-controller counts × 32 B) over wall clock, for STREAM and for the application alike. The harness (`perf/baseline`) is driven by matrix files and environment variables and runs through `mpirun` only.

Checks of the tools. The PAPI build costs nothing measurable (11.30 s against 11.34 s at 64×2; 521–523 s against 528 s on one core) and its per-kernel times add up to the wall clock within 1%; `MEM_ACCESS` equals `PAPI_L1_DCA` in every kernel; the cycle counter gives 2.59 GHz; the DDR counters were calibrated on STREAM (Section 2.4). The placement of every multi-rank run is checked by the counters: the busiest controller group carries 1.00× the mean.

Correctness gate. CloverLeaf's own check on the long deck `clover_bm.in` (960^2 cells, 2955 steps, test problem 3) reports a deviation of 0.57% (C) and 1.79% (Fortran) from the reference compiled into the code even for the *unmodified* kernels, identical to all digits at every layout and repetition (Table 1): the reference was produced on another machine. Every modified kernel therefore has to reproduce this baseline deviation to 10^{-6}% (`perf/baseline/qa_check.py`) and print `PASSED` on bm3840.

2.3 Where the time goes

Table 2 gives the time share and the counters of every kernel. The two advection kernels take 48% of the time on one core and 63% on 128; `advec_mom` scales worst of the large kernels (14× against 22–59×). On one core the IPC is 0.35–1.07 and 60–87% of the cycles are back-end stalls; at 64×2 the IPC is 0.10–0.50 per kernel and the stall share 81–96%: the cores wait for memory. Three kernel groups appear in the instruction mix: `pdv`, `accelerate` and `flux_calc` are vectorised (41–62% NEON); `calc_dt`, `ideal_gas` and `viscosity` run entirely scalar (they contain `sqrt`, which GCC keeps scalar without `-fno-math-errno`) but are only 12% of the node-level run; `advec_mom` and `advec_cell` are 12–16% vector instructions because their upwind loops have data-dependent indices. The `perf` profile of a 128-thread run adds that 12.5% of all cycles are spent in the OpenMP barrier (`gomp_team_barrier_wait_end`): every kernel is a sequence of worksharing loops with an implicit barrier each.

#	kernel	time share (%) 1 core	128 cores	IPC 1 core	vector ins. (%)	FP scalar ins. (%)	back-end stall (%)
1	`advec_mom`	28.6	44.0	0.85	16	17	71 / 94
2	`advec_cell`	19.4	19.4	0.75	12	25	73 / 93
3	`calc_dt`	11.8	4.8	0.65	0	64	75 / 84
4	`pdv_correct`	9.0	5.0	0.35	62	0	83 / 93
5	`pdv_predict`	8.6	4.6	0.35	60	0	87 / 95
6	`ideal_gas`	7.4	4.7	0.52	0	53	79 / 92
7	`viscosity`	5.7	2.2	1.06	0	64	60 / 79
8	`accelerate`	5.2	4.1	0.63	41	0	77 / 93
9	`flux_calc`	2.2	3.0	0.53	45	0	80 / 96
10	`reset_field`	1.1	3.2	0.56	0	0	80 / 97
11	`revert`	0.5	1.3	0.64	0	0	78 / 96
12	`field_summary`	0.4	0.2	0.64	56	24	72 / 90
13	`update_halo`	0.1	3.5	0.67	3	0	49 / 95

Table 2: Per-kernel time share (bm3840; "128 cores" = 1×128), IPC, share of SIMD and scalar floating-point instructions, and back-end stall cycles on 1 / 128 cores. Numbers refer to Figure 1. `reset_field`, `revert` and `update_halo` do no floating-point work.

2.4 Ceilings and byte model

Memory. STREAM triad (10^8 doubles, 20 iterations, 10 runs per configuration) sustains 15 GB/s on one core, 67 GB/s per NUMA domain (32 threads) and **266.5 GB/s** for the node, in every one of 10 pinned repetitions. Under the DDR-controller counters the controllers move 1.04× the bytes STREAM counts itself (24 B per triad element) and 0.74× the write-allocate model, in all 20 STREAM runs and in our own triad kernel: a streaming store does *not* read the line first on this CPU, one count is 32 B, and the hardware ceiling is the counted triad, 78% of the bus. (The DIMM clock is not readable without root; 266.5 GB/s over 16 channels needs at least 2082 MT/s, so DDR4-2666 is consistent.)

Compute. Independent FMA chains on 2-lane NEON registers reach 4.0 flop/cycle per core and **1330 GFLOP/s** per node; scalar FMA reaches the same, so vectorisation saves instructions, not flops. Vector division runs at 0.19 GFLOP/s per core. The ridge point of the node is 1330/266.5 = 5 flop/B (0.7 flop/B for one core).

Byte model. For every loop nest of the C kernels we counted the arrays read and written and the flops per cell; each array is streamed from DRAM once per loop nest, and a store is charged a read-for-ownership (write-allocate) in addition to the write. Table 3 gives 451 flop and 2186 B per cell and step (1688 B without the read-for-ownership), an intensity of 0.21 flop/B.

Validation. The model predicts 2.80 TB for bm3840; the counters measure 2.83 TB at 64×2 in all five repetitions (1% above), 2.79–2.99 TB at the other layouts. The read/write split decides which version of the model applies: the counters see 2.05 TB of reads and 0.77 TB of writes against 1.53 TB of demand reads and 0.64 TB of writes in the model, i.e. the reads are 1.34× the demand reads. Unlike STREAM, the C kernels' stores *do* cause a read of the line first, for about 70% of the written volume; the write-allocate model is the right one for the code as it stands. The Fortran build writes the same amount (0.60 TB) but reads only 1.15× the demand reads (1.76 TB, 2.36 TB in total): its loops avoid most of that traffic, and this 0.47 TB is the entire C/Fortran gap (Section 2.5).

kernel	calls/step	flop/cell	B/cell (no WA)	B/cell (with WA)	AI (flop/B, WA)
`ideal_gas`	2.1	9	32	48	0.19
`viscosity`	1	40	40	48	0.83
`calc_dt`	1	31	80	88	0.35
`pdv_predict`	1	33	96	120	0.28
`pdv_correct`	1	33	112	136	0.24
`revert`	1	0	32	48	0.00
`accelerate`	1	37	80	96	0.39
`flux_calc`	1	10	64	80	0.12
`advec_cell`	2	45	180	244	0.18
`advec_mom`	4	39	172	220	0.18
`reset_field`	1	0	64	96	0.00
`field_summary`	0.1	22	48	48	0.46
per cell and step		451	1688	2186	0.21

Table 3: Analytic flop and byte counts per cell and call (WA = write-allocate). `ideal_gas` runs 2.1 times per step.

kernels	layout	wall (s) median (min–max)	DRAM (TB)	bandwidth (GB/s)	busiest/ mean	vs. hw roof (%)
c	64x2	11.32 (11.24–11.33)	2.83	250	1.00	94
c	1x128	26.91 (17.44–29.20)	2.99	111	1.66	42
c	4x32	13.00 (12.76–13.21)	2.87	221	1.01	83
c	32x4	11.46 (11.41–11.47)	2.79	244	1.00	91
c	128x1	11.66 (11.63–11.77)	2.95	253	1.00	95
c	1x128, THP off	13.31 (13.28–13.92)	2.83	212	1.00	80
fortran	64x2	9.33 (9.33–9.42)	2.35	252	1.00	95
fortran	1x128	20.63 (18.40–25.92)	2.44	118	1.45	44
fortran	4x32	10.88 (10.78–11.22)	2.38	219	1.00	82
fortran	32x4	9.46 (9.44–9.52)	2.30	243	1.00	91
fortran	128x1	9.76 (9.66–9.77)	2.46	252	1.00	94

Table 4: Baseline layouts, bm3840, job 34797: median of 5 (min–max), hardware bytes of the DDR-controller counters, the resulting bandwidth and its share of the 266.5 GB/s ceiling, and the load of the busiest controller group relative to the mean. THP as installed except for the one marked row.

2.5 Baseline performance

Table 4 gives the layouts of the C kernels with the hardware bandwidth they sustain over the whole run, start-up included. **64×2 is the baseline**: 11.3 s (spread 0.8%), 2.83 TB, 250 GB/s = **94% of the hardware ceiling**; 32×4 and 128×1 reach 91–95%, 4×32 83%. Transparent huge pages (512 MB on this system) matter only for the single-rank 1×128 layout, where first touch places whole arrays on one NUMA domain and the busiest controller group carries 1.66× the mean (26.9 s, against 13.3 s with THP disabled for the process); with four or more ranks every rank touches its own data and THP has no effect, so the baseline needs no change to the system.

Figure 1 places every kernel on the rooflines of one core and of the node. Every kernel lies at 0.13–0.83 flop/B, far left of the 5 flop/B ridge; even `viscosity` could reach only 17% of the FMA peak with perfect bandwidth. In hardware-byte terms (per-section roofline of the 64×2 run, `perf/roofline/roofline.py -hw`) both advection sections, PdV, `ideal_gas` and the fluxes run at 94–102% of the memory roof, acceleration at 83%, and the scalar `sqrt` kernels at 63–66%. The floor at the measured traffic is 2.83 TB / 266.5 GB/s = **10.6 s**; the baseline is within 7%

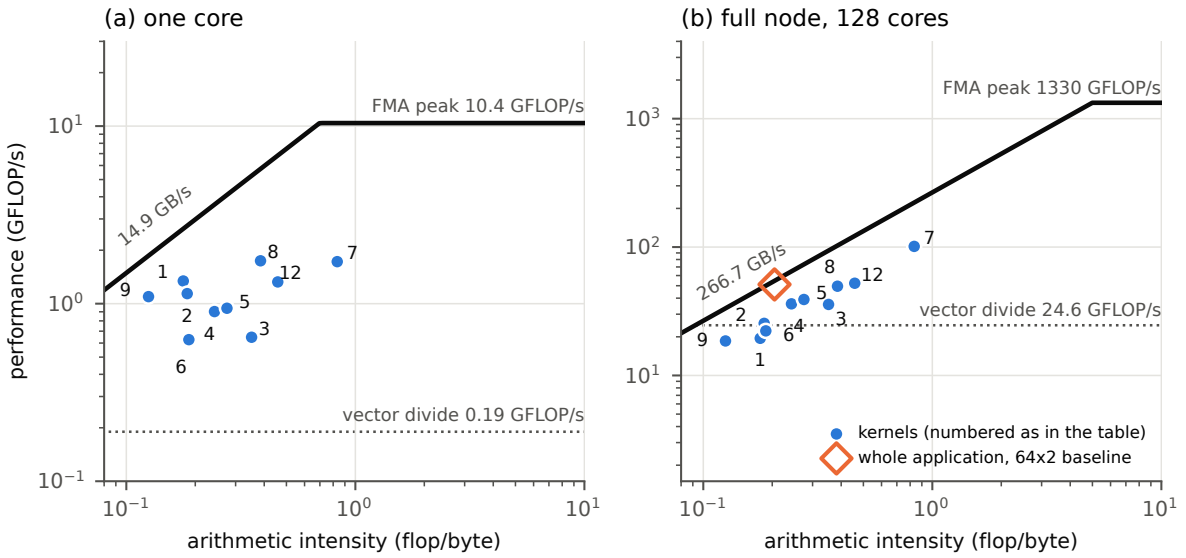


Figure 1: Roofline of the CloverLeaf C kernels (bm3840), one core and full node. Kernel numbers refer to Table 2; kernel points use model bytes and PAPI times (1×128), the application point the measured DRAM bytes and wall clock of the 64×2 baseline.

of it. There is no headroom at the current byte count: *every further gain has to come from moving fewer bytes*, and the Fortran build shows that this is possible (9.33 s, 2.36 TB, at the same 252 GB/s).

3 Kernel Improvements

3.1 What the analysis allows

The baseline sustains 94% of the memory ceiling, so instruction-side work (vectorising the upwind loops, `-fno-math-errno` for the `sqrt` kernels, fewer barriers) cannot pay off until the bytes shrink; those items are listed at the end of Table 5 as time-only candidates. The byte count has two components we can change: the bytes the algorithm asks for (temporaries, dead stores, copies, repeated helper loops) and the read-for-ownership that the C stores pay and a streaming store would not. Every change below is expressed as an edit of the loop table of the byte model (`perf/analysis/plan_bytes.py`), so the prediction and the later measurement use the same convention, and every change has to pass the correctness gate of Section 2.2.

3.2 Byte reductions

(1) `advec_mom`: direction-only helper loops once per sweep (−12.8%). The kernel is called four times per step (two sweeps × `xvel`, `yvel`). Its first four loops (`post_vol`/`pre_vol`, `node_flux`, `node_mass_post`, `node_mass_pre`) depend only on the sweep direction, not on the velocity component, and are recomputed for `yvel` with identical results. The kernel is split in a preparation part called once per sweep and a velocity part called twice:

change	B/cell (WA)	DRAM bytes	cumulative bytes	floor (s)
advec_mom: direction-only helper loops once per sweep	1906	−12.8%	−12.8%	9.17
advec_cell: keep 4 temporaries in registers	2058	−5.9%	−18.7%	9.90
drop dead stores post_vol / pre_vol	2090	−4.4%	−21.6%	10.06
Reset/Revert copies by pointer swap	2042	−6.6%	−28.2%	9.83
calc_dt: fuse the min reduction	2162	−1.1%	−29.3%	10.41
all byte changes together	1546	−29.3%	−29.3%	7.44
streaming stores (no read-for-ownership), upper bound	1688	−22.8%	−41.4%	8.13
nowait between independent worksharing loops	time only: 12.5 % of cycles in gomp barrier (perf, 1x128)			
-fno-math-errno for the sqrt kernels	time only: calc_dt, ideal_gas, viscosity run scalar (0 % NEON)			
omp simd / restructured upwind indexing in advection flux sweeps	time only: only after the byte reductions: at 64x2 both C and Fortran run at 94-95 % of the hardware ceiling			
tile-level parallelism	time only: one fork/join per tile instead of per loop (fallback if nowait is not enough)			

Table 5: Kernel changes and their predicted effect on hardware bytes (write-allocate model, bm3840, 87 steps; floor at 266.5 GB/s; measured baseline 2.83 TB, 11.3 s). Cumulative = all byte changes up to that row; for the streaming-store bound, cumulative = on top of all five.

```
! advec_mom_driver.f90, before: one call per velocity component recomputes
! post_vol, pre_vol, node_flux, node_mass_post, node_mass_pre
CALL advec_mom_kernel_c(..., xvel1, ..., which_vel=1, sweep_number)
CALL advec_mom_kernel_c(..., yvel1, ..., which_vel=2, sweep_number)
! after: the helpers once, the upwind flux and the velocity update per component
CALL advec_mom_prep_c(..., node_flux, node_mass_post, node_mass_pre, &
                      direction, sweep_number)
CALL advec_mom_vel_c (..., xvel1, node_flux, node_mass_pre, direction)
CALL advec_mom_vel_c (..., yvel1, node_flux, node_mass_pre, direction)
```

(2) `advec_cell`: temporaries in registers (−5.9%). The update loop of each sweep stores `pre_mass`, `post_mass`, `post_ener` and `advec_vol` to four arrays and reads them back in the same iteration; nothing else reads them. They become scalars (no store, no read-for-ownership):

```
/* before (x sweep; i = FTNREF2D(j,k,...)) */
pre_mass[i]  = density1[i]*pre_vol[i];
post_mass[i] = pre_mass[i] + mass_flux_x[i] - mass_flux_x[i+1];
post_ener[i] = (energy1[i]*pre_mass[i] + ener_flux[i] - ener_flux[i+1])
               / post_mass[i];
advec_vol[i] = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
density1[i]  = post_mass[i] / advec_vol[i];
energy1[i]   = post_ener[i];
/* after */
double pre_mass  = density1[i]*pre_vol[i];
double post_mass = pre_mass + mass_flux_x[i] - mass_flux_x[i+1];
double advec_vol = pre_vol[i] + vol_flux_x[i] - vol_flux_x[i+1];
energy1[i]  = (energy1[i]*pre_mass + ener_flux[i] - ener_flux[i+1]) / post_mass;
density1[i] = post_mass / advec_vol;
```

(3) Dead stores (−4.4%). `post_vol` in the first sweep of `advec_cell` and `pre_vol` in `advec_mom` are written over the whole (haloed) grid and never read; the stores are removed:

```
/* advec_cell, sweep 1 */
pre_vol[i]  = volume[i] + (vol_flux_x[i+1]-vol_flux_x[i])
                        + (vol_flux_y[iy+1]-vol_flux_y[iy]);
post_vol[i] = pre_vol[i] - (vol_flux_x[i+1]-vol_flux_x[i]);   /* removed: never read */
/* advec_mom, every sweep */
post_vol[i] = volume[i] + vol_flux_y[iy+1] - vol_flux_y[iy];
pre_vol[i]  = post_vol[i] + vol_flux_x[i+1] - vol_flux_x[i];  /* removed: never read */
```

(4) Reset and Revert without copies (−6.6%). `reset_field` copies the four end-of-step fields (`density1`, `energy1`, `xvel1`, `yvel1`) into their level-0 partners, 118 MB each; `revert` copies `density0`/`energy0` back into the level-1 arrays before the PdV corrector because the predictor wrote into them. The fields are allocatable components of the tile, so the reset becomes a swap of the allocations, and the revert disappears because the C corrector reads only the level-0 fields:

```
! reset_field.f90: replaces the four copy loops of reset_field_kernel_c
CALL swap(field%density0, field%density1);  CALL swap(field%energy0, field%energy1)
CALL swap(field%xvel0,    field%xvel1);     CALL swap(field%yvel0,   field%yvel1)

SUBROUTINE swap(a, b)                          ! O(1), no traffic
  REAL(KIND=8), ALLOCATABLE, INTENT(INOUT) :: a(:,:), b(:,:)
  REAL(KIND=8), ALLOCATABLE :: t(:,:)
  CALL MOVE_ALLOC(a, t); CALL MOVE_ALLOC(b, a); CALL MOVE_ALLOC(t, b)
END SUBROUTINE
! PdV.f90: the CALL revert() at the end of the predictor pass is dropped
```

(5) `calc_dt`: fused reduction (−1.1%). The kernel stores the local time step of every cell to `dt_min` and reduces the array in a second loop. The reduction moves into the first loop; the cell of the minimum, needed only in the failure message, is located afterwards by a scan over the scalar criterion when the step is too small:

```
/* before */
#pragma omp for private(...)
for (k...) for (j...) { ... dt_min[i] = MIN(dtct, MIN(dtut, MIN(dtvt, dtdivt))); }
#pragma omp for reduction(min:dt_min_val)
for (k...) for (j...) if (dt_min[i] < dt_min_val) dt_min_val = dt_min[i];
/* after: no dt_min array */
#pragma omp for private(...) reduction(min:dt_min_val)
for (k...) for (j...) {
    ... dt_min_val = MIN(dt_min_val, MIN(dtct, MIN(dtut, MIN(dtvt, dtdivt)))); }
```

(6) Streaming stores (bound −23%, −41% with the five above). Section 2.4 showed that a streaming store moves no read-for-ownership bytes on this CPU while the C kernels' stores do, and that the Fortran build already avoids most of them. Two routes are open: making the store streams of a loop recognisable to the core's write-streaming detector (every output array written once per iteration, contiguously, without loads from the same lines, which also follows from changes 2 and 3), or explicit non-temporal stores where the output is never re-read in the loop:

```
/* candidate: the velocity update of advec_mom (vel1 read once, written once) */
#pragma omp simd
for (j = x_min; j <= x_max+1; j++) {
    double v = (vel1[i]*node_mass_pre[i] + mom_flux[i-1] - mom_flux[i])
               / node_mass_post[i];
    __builtin_nontemporal_store(v, &vel1[i]);   /* clang; GCC: inline stnp */
}
```

The no-write-allocate model is only an upper bound for this item; the counters decide what the hardware actually grants.

layout	baseline			optimised			speed-up
	wall (s)	TB	GB/s	wall (s)	TB	GB/s	
64x2	11.32 (11.24–11.33)	2.83	250	–	–	–	–
1x128	26.91 (17.44–29.20)	2.99	111	–	–	–	–
4x32	13.00 (12.76–13.21)	2.87	221	–	–	–	–
32x4	11.46 (11.41–11.47)	2.79	244	–	–	–	–
128x1	11.66 (11.63–11.77)	2.95	253	–	–	–	–

Table 6: C kernels, bm3840: baseline (job 34797) against the optimised build at every layout, THP as installed. Speed-up = baseline median / optimised median.

3.3 Time-only changes

After the bytes shrink, three items move the kernels off the memory roof: `nowait` between independent worksharing loops (12.5% of the cycles are barrier time), `-fno-math-errno` so that the `sqrt` kernels vectorise, and an `omp simd`-friendly formulation of the upwind loops (select the donor index arithmetically instead of by branch). Tile-level parallelism (one fork/join per tile instead of per loop) is the fallback if `nowait` is not enough.

3.4 Implementation status

[TODO: placeholder: for each change, implemented (yes/no), passes the correctness gate, measured bytes and time at 64×2, link to the commit. Table 6 and Figure 2 are filled from `data/optimised.csv` once the first change is measured.]

4 Results

Every optimised build is measured exactly like the baseline (Section 2.2): five repetitions under the DDR-controller counters, at the same five layouts, the correctness gate on `clover_bm.in` and `PASSED` on bm3840. Table 6 compares every layout with its own baseline; Figure 2 shows the wall clock against the memory floor.

[TODO: results text: bytes removed per change against the prediction of Table 5, speed-up per layout, updated roofline position, remaining gap to the floor.]

5 Conclusions (draft)

- The C kernels are memory-bound (0.2 flop/B against a 5 flop/B ridge) and the 64×2 baseline already sustains 94% of the 266.5 GB/s hardware ceiling over the whole run: only fewer bytes can make it faster.
- The analytic byte model matches the counters to 1% and predicts the five planned changes at −29% of the traffic (floor 10.5 to 7.4 s); the read-for-ownership that the C stores pay, and that streaming stores and the Fortran build avoid, is a sixth lever worth up to another −23%.
- The correctness gate (reproduction of the `clover_bm.in` deviation) held at every layout for the baseline and is applied to every change.

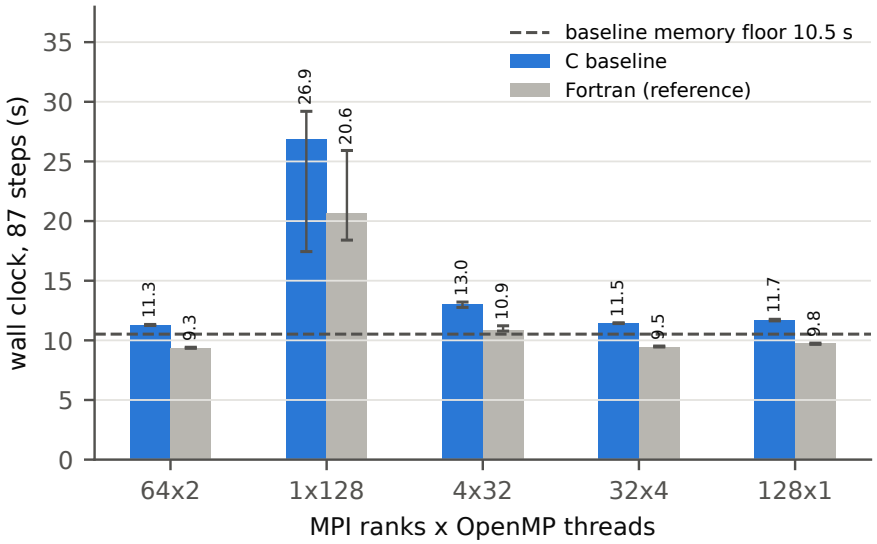


Figure 2: Wall clock per layout (median, bars min–max): C baseline, C optimised where measured, Fortran for reference. The dashed line is the memory floor of the baseline byte count at 266.5 GB/s.

References

[1] Jan Laukemann, Georg Hager, Gerhard Wellein, et al. CloverLeaf on Intel multi-core CPUs: A case study in write-allocate evasion. arXiv:2311.04797, 2023.

[2] A. C. Mallinson, D. A. Beckingsale, W. P. Gaudin, J. A. Herdman, J. M. Levesque, and S. A. Jarvis. CloverLeaf: Preparing hydrodynamics codes for exascale. In *Proceedings of the Cray User Group (CUG)*, 2013.